

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240191 123240193
NAMA	:	Yohanes Herang Aji Dharma Alvin Andhika Putra
PLUG	:	IF-D
NAMA ASISTEN	:	Hawla Khufyatul Qolbu Akfina Ni'mawati

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Yohanes Herang Aji Dharma 123240191
Alvin Andhika Putra 123240193

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Hawla Khufyatul Qolbu
NIM. 123230021

Asisten Praktikum

Akfina Ni'mawati
NIM. 123230076

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pendukung Keputusan Pemilihan Tempat Magang Berdasarkan Benefit dan Lokasi . Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 25 Mei 2026

Penyusun 1,

Penyusun 2,

Alvin Andhika Putra

NIM. 123240193

Yohaness Herang Aji Dharma

NIM. 123240191

DAFTAR ISI

HALAMAN PENGESAHAN.....	2
KATA PENGANTAR.....	3
DAFTAR ISI.....	4
BAB I	
PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	2
BAB II	
PEMBAHASAN.....	3
2.1 Dasar Teori.....	3
2.2 Skenario Kasus & Dataset.....	6
2.3 Pemodelan Sistem Pendukung Keputusan.....	6
2.4 Perhitungan & Algoritma.....	8
2.5 Implementasi & Tampilan Sistem.....	11
BAB III	
JADWAL Pengerjaan dan Pembagian Tugas.....	27
3.1 Jadwal Pengerjaan.....	27
3.2 Pembagian Tugas.....	27
BAB IV	
KESIMPULAN DAN SARAN.....	29
4.2 Kesimpulan.....	29
4.3 Saran.....	29
DAFTAR PUSTAKA.....	30

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Magang merupakan salah satu kegiatan yang penting bagi mahasiswa untuk memperoleh pengalaman kerja serta meningkatkan keterampilan sebelum memasuki dunia kerja yang sebenarnya. Saat ini terdapat banyak lowongan magang yang ditawarkan oleh berbagai perusahaan dengan benefit, lokasi, durasi, dan bidang pekerjaan yang berbeda-beda. Banyaknya pilihan tersebut sering kali membuat mahasiswa mengalami kesulitan dalam menentukan tempat magang yang paling sesuai dengan kebutuhan dan keinginan mereka.

Dalam proses pemilihan tempat magang, mahasiswa biasanya mempertimbangkan beberapa faktor seperti besarnya stipend atau gaji, lokasi magang, sistem kerja (remote atau onsite), durasi magang, serta bidang pekerjaan yang diminati. Namun, proses pemilihan tersebut sering dilakukan secara manual sehingga membutuhkan waktu yang cukup lama dan terkadang menghasilkan keputusan yang kurang optimal. Selain itu, adanya data lowongan yang banyak dan beragam juga dapat menyebabkan mahasiswa kesulitan dalam membandingkan setiap pilihan magang secara objektif.

Permasalahan tersebut dapat diselesaikan dengan memanfaatkan Sistem Pendukung Keputusan (SPK). Sistem Pendukung Keputusan dapat membantu pengguna dalam melakukan proses seleksi dan penilaian berdasarkan beberapa kriteria tertentu sehingga keputusan yang dihasilkan menjadi lebih efektif dan terstruktur. Dengan adanya sistem ini, pengguna dapat mengetahui tempat magang yang memiliki benefit terbaik sesuai dengan kriteria yang diinginkan.

Pada proyek ini akan dibuat sebuah Sistem Pendukung Keputusan untuk menentukan tempat magang terbaik berdasarkan beberapa kriteria seperti benefit atau stipend, lokasi, dan durasi magang. Sistem ini diharapkan dapat membantu mahasiswa dalam memilih tempat magang secara lebih cepat, tepat, dan efisien.

1.2 Tujuan Proyek Akhir

Tujuan dari proyek akhir ini adalah untuk merancang dan membangun Sistem Pendukung Keputusan yang dapat membantu mahasiswa dalam menentukan tempat magang terbaik berdasarkan beberapa kriteria tertentu. Sistem ini dibuat agar proses pemilihan tempat magang dapat dilakukan secara lebih mudah, cepat, dan tepat.

Adapun tujuan yang ingin dicapai dalam proyek akhir ini adalah sebagai berikut:

- Membuat sistem yang dapat mengelola data lowongan magang dari file CSV.
- Membantu pengguna dalam memilih tempat magang berdasarkan kriteria seperti benefit atau stipend, lokasi, dan durasi magang.
- Mengelompokkan data magang menjadi beberapa kategori agar lebih mudah dianalisis.

- Mengurangi kesalahan dalam proses pemilihan tempat magang yang dilakukan secara manual.
- Menerapkan metode Sistem Pendukung Keputusan untuk menghasilkan rekomendasi tempat magang terbaik.
- Memberikan hasil rekomendasi yang lebih efektif dan efisien bagi mahasiswa dalam menentukan pilihan tempat magang.

1.3 Manfaat Proyek Akhir

Proyek akhir ini diharapkan dapat memberikan manfaat bagi mahasiswa maupun pengguna yang ingin mencari tempat magang yang sesuai dengan kebutuhan dan kriteria tertentu. Dengan adanya sistem ini, proses pemilihan tempat magang yang sebelumnya dilakukan secara manual dapat menjadi lebih mudah, cepat, dan terstruktur. Pengguna tidak perlu lagi membandingkan banyak lowongan satu per satu karena sistem akan membantu memberikan rekomendasi berdasarkan data dan kriteria yang telah ditentukan.

Selain itu, sistem ini juga dapat membantu pengguna dalam mengetahui tempat magang yang memiliki benefit terbaik, baik dari segi stipend atau gaji, lokasi, maupun durasi magang. Dengan adanya pengelompokan data dan proses perhitungan menggunakan metode Sistem Pendukung Keputusan, hasil rekomendasi yang diberikan diharapkan menjadi lebih efektif dan dapat membantu pengguna dalam mengambil keputusan yang lebih tepat.

Bagi mahasiswa, proyek ini juga dapat menjadi sarana untuk memahami penerapan Sistem Cerdas dan Pendukung Keputusan dalam menyelesaikan permasalahan nyata di kehidupan sehari-hari. Melalui proyek ini, mahasiswa dapat mempelajari bagaimana cara mengolah data, melakukan preprocessing dataset, mengelompokkan data, hingga menghasilkan sebuah sistem rekomendasi yang dapat digunakan secara langsung.

Selain memberikan manfaat dalam proses pemilihan tempat magang, proyek ini juga dapat dijadikan sebagai referensi untuk pengembangan sistem serupa di masa mendatang, seperti sistem rekomendasi pekerjaan, beasiswa, maupun pemilihan program pelatihan berdasarkan beberapa kriteria tertentu.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Sistem Pendukung Keputusan atau Decision Support System (DSS) merupakan suatu sistem yang digunakan untuk membantu pengguna dalam mengambil keputusan berdasarkan data dan kriteria tertentu. Sistem ini biasanya digunakan untuk menyelesaikan permasalahan yang memiliki banyak pilihan dan memerlukan proses penilaian agar menghasilkan keputusan yang lebih tepat, efektif, dan efisien.

Dalam proyek akhir ini, Sistem Pendukung Keputusan digunakan untuk membantu mahasiswa dalam menentukan tempat magang terbaik berdasarkan beberapa kriteria seperti benefit atau stipend, lokasi, dan durasi magang. Dengan adanya sistem ini, proses pemilihan tempat magang dapat dilakukan secara lebih terstruktur dibandingkan dengan cara manual.

Metode Simple Additive Weighting (SAW)

Metode Simple Additive Weighting (SAW) merupakan salah satu metode yang paling sering digunakan dalam Sistem Pendukung Keputusan. Metode ini dikenal juga sebagai metode penjumlahan terbobot karena proses perhitungannya dilakukan dengan menjumlahkan nilai setiap kriteria yang telah dikalikan dengan bobot masing-masing.

Metode SAW bekerja dengan cara mencari nilai penjumlahan dari rating kinerja setiap alternatif pada semua atribut atau kriteria. Alternatif dengan nilai akhir terbesar akan menjadi pilihan terbaik karena dianggap paling sesuai dengan kriteria yang telah ditentukan.

Pada proyek ini, metode SAW digunakan untuk menentukan tempat magang terbaik berdasarkan beberapa kriteria yang telah dipilih. Setiap tempat magang akan diberikan nilai sesuai dengan kriteria tertentu, kemudian dilakukan proses normalisasi dan perhitungan bobot untuk menghasilkan ranking akhir.

Langkah-Langkah Metode SAW

Berikut merupakan tahapan dalam metode SAW:

1. Menentukan Kriteria

Kriteria merupakan faktor yang digunakan dalam proses penilaian. Pada proyek ini, kriteria yang digunakan antara lain:

- Benefit atau stipend
- Lokasi magang
- Durasi magang

2. Menentukan Bobot Kriteria

Setiap kriteria memiliki tingkat kepentingan yang berbeda sehingga perlu diberikan bobot. Contoh:

- Benefit = 50%
- Lokasi = 30%
- Durasi = 20%

Bobot tersebut digunakan untuk menentukan seberapa besar pengaruh suatu kriteria terhadap hasil akhir.

3. Membuat Matriks Keputusan

Matriks keputusan berisi nilai dari setiap alternatif berdasarkan masing-masing kriteria.

Alternatif	Benefit	Lokasi	Durasi
A1	90	80	70
A2	85	90	75
A3	70	85	95

4. Normalisasi Matriks

Proses normalisasi dilakukan agar seluruh nilai kriteria memiliki skala yang sama sehingga dapat dibandingkan dengan lebih mudah.

Untuk kriteria benefit digunakan rumus:

$$r_{ij} = \frac{x_{ij}}{\max(x_{ij})}$$

Keterangan:

- r_{ij} = Nilai Hasil Normalisasi
- x_{ij} = Nilai Asli Alternatif
- $\max(x_{ij})$ = Nilai Terbesar Pada Suatu Kriteria

Sedangkan untuk kriteria cost digunakan rumus:

$$r_{ij} = \frac{\min(x_{ij})}{x_{ij}}$$

Keterangan :

- r_{ij} = Nilai Hasil Normalisasi
- x_{ij} = Nilai Asli Alternatif
- $\min(x_{ij})$ = Nilai Terkecil Pada Suatu Kriteria

5. Menghitung Nilai Preferensi

Nilai akhir diperoleh dari hasil penjumlahan nilai normalisasi yang telah dikalikan dengan bobot masing-masing kriteria.

Rumus nilai preferensi:

$$V_i = \sum_{j=1}^n w_j r_{ij}$$

Keterangan:

- V_i = Nilai Akhir Alternatif
- w_j = Bobot Kriteria
- r_{ij} = Hasil Normalisasi

Alternatif dengan nilai terbesar akan dipilih sebagai rekomendasi terbaik.

2.2 Skenario Kasus & Dataset

Pada era saat ini , mahasiswa sering mengalami kesulitan dalam menentukan tempat magang terbaik karena banyaknya pilihan perusahaan yang tersedia. Setiap perusahaan memiliki kelebihan dan kekurangannya masing masing , seperti lokasi magang , gaji magang , durasi magang , fasilitas tambahan ,dan tipe kerja yang ditawarkan (WFH / WFO). Oleh karena itu , diperlukan sebuah Sistem Pendukung Keputusan untuk membantu mahasiswa memilih tempat magang yang terbaik.

Pada project akhir ini , pengambilan keputusan adalah mahasiswa yang sedang mencari tempat magang terbaik . Tujuan utama program ini adalah membantu mahasiswa menentukan alternatif tempat magang terbaik berdasarkan beberapa kriteria penilaian menggunakan metode Simple Additive Weighting (SAW)

Dataset yang digunakan berasal dari data lowongan magang yang terdapat pada situs internet kaggle bernama merged_internsip_dataset.csv. Dataset tersebut berisi informasi mengenai perusahaan dan detail lowongan magang seperti :

- Nama Perusahaan
- Posisi Magang
- Lokasi
- Gaji Magang
- Durasi Magang
- Fasilitas Tambahan / Perks

2.3 Pemodelan Sistem Pendukung Keputusan

1. Kriteria

Kriteria	Jenis
Location	Cost
Stipend	Benefit
Duration	Benefit
Perks	Benefit
Tipe Kerja	Benefit

Penjelasan :

- **Cost = Semakin Kecil Nilainya Semakin Baik**
- **Benefit = Semakin Besar Nilainya Semakin Baik**

2. Alternatif

- A1 = Splashgain Technology Solutions Private Limited
- A2 = Icecreamlabs
- A3 = Entrepreneur Growth Lab (OPC) Private Limited

3. Skala Penilaian

a. Location (Cost)

Nilai	Keterangan
1	Sangat Dekat
5	Sedang
10	Sangat Jauh

b. Stipend (Benefit)

Nilai	Keterangan
1	Sangat Kecil
10	Sangat Besar
5	Sedang

c. Duration (Benefit)

Nilai	Keterangan
10	Sangat Lama
5	Sedang
5	Sedang

d. Perks (Benefit)

Nilai	Keterangan
10	Sangat Lengkap
1	Sedikit
5	Sedang

e. Tipe Kerja (Benefit)

Nilai	Keterangan
7	WFO
7	WFO
10	WFH

4. Matriks Keputusan

Alternatif	Location	Stipend	Duration	Perks	Tipe Kerja
A1	1	1	10	10	7
A2	5	10	5	1	7
A3	10	5	5	5	10

2.4 Perhitungan & Algoritma

Kriteria	Bobot
Location	30%
Stipend	30%
Duration	10%
Perks	10%
Tipe Kerja	20%

Langkah 1 : Normalisasi Matriks

Rumus Cost :

$$r_{ij} = \frac{\min(x_{ij})}{x_{ij}}$$

- Location (Cost)
 - Nilai Minimum = 1
 - $A1 = \frac{1}{1} = 1.00$
 - $A2 = \frac{1}{5} = 0.20$
 - $A3 = \frac{1}{10} = 0.10$

Rumus Benefit :

$$r_{ij} = \frac{x_{ij}}{\max(x_{ij})}$$

- Stipend (Benefit)
 - Nilai Maksimum = 10
 - $A1 = \frac{1}{10} = 0.10$
 - $A2 = \frac{10}{10} = 1.00$
 - $A3 = \frac{5}{10} = 0.50$
- Duration (Benefit)
 - Nilai Maksimum = 10
 - $A1 = \frac{10}{10} = 1.00$
 - $A2 = \frac{5}{10} = 0.50$
 - $A3 = \frac{5}{10} = 0.50$
- Perks (Benefit)
 - Nilai Maksimum = 10
 - $A1 = \frac{10}{10} = 1.00$
 - $A2 = \frac{1}{10} = 0.10$
 - $A3 = \frac{5}{10} = 0.50$
- Tipe Kerja (Benefit)
 - Nilai Maksimum = 10
 - $A1 = \frac{7}{10} = 0.70$
 - $A2 = \frac{7}{10} = 0.70$
 - $A3 = \frac{10}{10} = 1.00$

Langkah 2 : Perhitungan Nilai Preferensi

Bobot :

- Location = 30%
- Stipend = 30%
- Duration = 10%
- Perks = 10%
- Tipe Kerja = 20%

Perhitungan A1

$$V1 = (1.00 \times 0.30) + (0.10 \times 0.30) + (1.00 \times 0.10) + (1.00 \times 0.10) + (0.70 \times 0.20)$$

$$V1 = 0.30 + 0.03 + 0.10 + 0.10 + 0.14$$

$$V1 = 0.67$$

Perhitungan A2

$$V2 = (0.20 \times 0.30) + (1.00 \times 0.30) + (0.50 \times 0.10) + (0.10 \times 0.10) + (0.70 \times 0.20)$$

$$V2 = 0.06 + 0.30 + 0.05 + 0.01 + 0.14$$

$$V2 = 0.56$$

Perhitungan A3

$$V3 = (0.10 \times 0.30) + (0.50 \times 0.30) + (0.50 \times 0.10) + (0.50 \times 0.10) + (1.00 \times 0.20)$$

$$V3 = 0.03 + 0.15 + 0.05 + 0.05 + 0.20$$

$$V3 = 0.48$$

Hasil Perankingan

Ranking	Alternatif	SAW
1	A1	0.67
2	A2	0.56
3	A3	0.48

2.5 Implementasi & Tampilan Sistem

Source Code Internship.py

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from Style import *

# Untuk Judul Interface
st.set_page_config(page_title="Project SCPK - Menentukan Lokasi Magang Terbaik", layout="wide")
st.markdown(visualisasi(),unsafe_allow_html=True)

# Untuk Header
st.title("Program Pengambilan Keputusan Tempat Magang Terbaik")
st.subheader("Metode SAW")
st.write(
    "Anggota Kelompok : \n"
    "1. Yohanes Herang Aji Dharma || 123240191 \n"
    "2. Alvin Andhika Putra || 123240193"
)

# Untuk Tab
Tab1, Tab2, Tab3, Tab4, Tab5, Tab6 = st.tabs([
    "Pilih Lowongan Magang", "Alternatif", "Skala Kriteria",
    "Pembobotan Kriteria", "Hasil Alternatif Terbaik", "Visualisasi"
])

df = pd.read_csv('merged_internships_dataset.csv')

with Tab1:

    profile_company_count = df.groupby('profile')['company'].nunique()

    # Filter profile yang memiliki 3 perusahaan atau lebih
    valid_profiles = profile_company_count[profile_company_count >= 3].index.tolist()
    valid_profiles.sort()

    options = ["Semua Lowongan"] + valid_profiles

    pilih_lowongan = st.selectbox(
        "Pilih Lowongan Magang:",
        options=options,
        key="selected_profile"
    )

    # Tampilkan jumlah lowongan yang ditemukan
    if pilih_lowongan == "Semua Lowongan":
        filtered_df_Tab1 = df
        st.info(f"Menampilkan semua {len(filtered_df_Tab1)} lowongan magang")
    else:
        filtered_df_Tab1 = df[df['profile'] == pilih_lowongan]
        st.success(f"Menampilkan {len(filtered_df_Tab1)} lowongan untuk posisi {pilih_lowongan}")
```

```

with Tab2 :
    if st.session_state.get('selected_profile') is None or
st.session_state.get('selected_profile') == "Semua Lowongan":
        st.warning("Harap Memilih Lowongan Magang Terlebih Dahulu Di tab
1")

    else:
        jumlah_lowongan_tersedia = len(filtered_df_Tab1)

        if jumlah_lowongan_tersedia > 3:
            max_slider = jumlah_lowongan_tersedia
            default_value = min(3, max_slider) # default 3 atau jumlah yang
tersedia jika kurang

            jumlahAlternatif = st.slider(
                "Masukkan Jumlah Alternatif :
",min_value=3,max_value=max_slider,value=default_value,step=1,disabled=Fa
lse)

            st.session_state.jumlah_alternatif = jumlahAlternatif
            daftar_perusahaan = filtered_df_Tab1['company'].unique().tolist()

            perusahaan_terpilih = st.multiselect(
                "Pilih Perusahaan untuk Alternatif",
                options=daftar_perusahaan,
                key="multiselect_perusahaan",
                max_selections=jumlahAlternatif
            )

            st.session_state.pilihan_perusahaan = perusahaan_terpilih.copy()

            for i in range(jumlahAlternatif):
                if i < len(perusahaan_terpilih):
                    nilai = perusahaan_terpilih[i]
                else:
                    nilai = "(belum dipilih)"
                hasilData(nilai,"Alternatif ",i+1)
                alt = nilai

            elif jumlah_lowongan_tersedia == 3:
                st.info(f"Hanya tersedia {jumlah_lowongan_tersedia} lowongan,
akan dipilih semua sebagai alternatif secara otomatis")

                daftar_perusahaan =
filtered_df_Tab1['company'].unique().tolist()
                jumlahAlternatif = 3 # Otomatis 3

                st.write(f"Alternatif yang tersedia:")

                pilihan_perusahaan = []
                for i, perusahaan in enumerate(daftar_perusahaan):
                    hasilDataBaru(perusahaan, "Alternatif ", i+1)
                    pilihan_perusahaan.append(perusahaan)

                # Simpan ke session state

```



```

        st.session_state.pilihan_perusahaan = pilihan_perusahaan
        st.session_state.jumlah_alternatif = jumlahAlternatif

with Tab3:
    selected_profile = st.session_state.get('selected_profile')
    pilihan_perusahaan = st.session_state.get('pilihan_perusahaan')

    if selected_profile is None or selected_profile == "Semua Lowongan":
        st.warning("Harap Memilih Lowongan Magang Terlebih Dahulu Di Tab
1")

    elif not pilihan_perusahaan or len(pilihan_perusahaan) == 0:
        st.warning("Harap Memilih Alternatif Perusahaan Terlebih Dahulu
Di Tab 2")
    else:
        st.subheader("Penentuan Skala Kriteria")

        # Ambil data perusahaan yang dipilih
        df_alternatif =
filtered_df_Tab1[filtered_df_Tab1['company'].isin(pilihan_perusahaan)]

        # Inisialisasi session state untuk menyimpan nilai skala
        if 'skala_kriteria' not in st.session_state:
            st.session_state.skala_kriteria = {}

        # Daftar kriteria
        kriteria_list = ["Location", "Stipend", "Duration", "Perks",
"Tipe Kerja"]

        jumlah_alternatif = len(pilihan_perusahaan)

        # Header tabel: Nama Alternatif (Perusahaan)
        # Kolom pertama untuk Kriteria, sisanya untuk setiap alternatif
        lebar_kolom = [1.5] + [1] * jumlah_alternatif
        cols_header = st.columns(lebar_kolom)
        with cols_header[0]:
            st.markdown("***Kriteria***")

        # Tampilkan nama perusahaan sebagai header kolom
        # enumerate : fungsi untuk memberi index (i) dan nilai
(perusahaan) secara bersamaan
        # pilihan perusahaan : ["A","B","C"] , iterasi 1 i=0 ,
perusahaan="A"
        for i, perusahaan in enumerate(pilihan_perusahaan):

            # kolom [0] untuk kriteria
            # kolom [1 -> n] untuk alternatif 1 -> n
            with cols_header[i + 1]:
                st.markdown(f"***Alternatif {i+1}***")
                # perusahaan[:20] itu buat mengambil 20 karakter nama
perusahaan
                # kalo lebih dari 20 karakter nanti di belakang ditambah
(...)
                st.caption(perusahaan[:20] + "..." if len(perusahaan) >
20 else perusahaan)

            # Looping untuk setiap kriteria
            for kriteria in kriteria_list:

```

```

# Buat kolom dinamis untuk setiap baris kriteria
lebar_kolom = [1.5] + [1] * jumlah_alternatif
cols = st.columns(lebar_kolom)

with cols[0]:
    with st.expander(f"{kriteria}"):
        for j, perusahaan in enumerate(pilihan_perusahaan):
            row = df_alternatif[df_alternatif['company'] ==
perusahaan]

            if not row.empty:
                if kriteria == "Location":
                    nilai = row['Location'].iloc[0]
                elif kriteria == "Stipend":
                    nilai = row['Stipend'].iloc[0]
                elif kriteria == "Duration":
                    nilai = row['Duration'].iloc[0]
                elif kriteria == "Perks":
                    nilai = row['Perks'].iloc[0]
                elif kriteria == "Tipe Kerja":
                    lokasi = row['Location'].iloc[0]
                    nilai = "WFH" if "Work from home" in
str(lokasi) else ("WFO" if str(lokasi) not in ["N/A", "Not Available"]
else "Tidak diketahui")
                else:
                    nilai = "N/A"
                st.write(f"**{perusahaan}** {nilai}")

# Input skala untuk setiap alternatif (1-10)
for j, perusahaan in enumerate(pilihan_perusahaan):
    with cols[j + 1]:
        skala_kriteria = f"skala_{kriteria}_{perusahaan}"
        default_value =
st.session_state.skala_kriteria.get(skala_kriteria, 1)

        skala = st.number_input(
            f"Skala {kriteria} - {perusahaan}",
            min_value=1,
            max_value=10,
            value=int(default_value),
            step=1,
            key=skala_kriteria,
            label_visibility="collapsed"
        )
        st.session_state.skala_kriteria[skala_kriteria] =
skala

with Tab4:
    selected_profile = st.session_state.get('selected_profile')
    pilihan_perusahaan = st.session_state.get('pilihan_perusahaan')
    skala_kriteria = st.session_state.get('skala_kriteria', {})

    if selected_profile is None or selected_profile == "Semua Lowongan":
        st.warning("Harap Memilih Lowongan Magang Terlebih Dahulu Di Tab
1")

    elif not pilihan_perusahaan or len(pilihan_perusahaan) == 0:
        st.warning("Harap Memilih Alternatif Perusahaan Terlebih Dahulu
Di Tab 2")

```

```

elif not skala_kriteria:
    st.warning("Harap Mengisi Skala Kriteria Terlebih Dahulu Di Tab
3")

else:
    st.caption("Atur bobot untuk setiap kriteria (total harus 100%)")

    # Daftar kriteria beserta jenisnya (Cost/Benefit)
    kriteria_list = [
        {"nama": "Location", "jenis": "Cost"},
        {"nama": "Stipend", "jenis": "Benefit"},
        {"nama": "Duration", "jenis": "Benefit"},
        {"nama": "Perks", "jenis": "Benefit"},
        {"nama": "Tipe Kerja", "jenis": "Benefit"}
    ]

    # Inisialisasi session state untuk bobot
    if 'bobot_kriteria' not in st.session_state:
        st.session_state.bobot_kriteria = {}

    # Tampilkan input bobot untuk setiap kriteria
    for kriteria in kriteria_list:
        col1, col2, col3 = st.columns([2, 1, 2])

        with col1:
            if kriteria["jenis"] == "Cost":
                st.markdown(f"**{kriteria['nama']}**")
            else:
                st.markdown(f"**{kriteria['nama']}**")

        with col2:
            key_bobot = f"bobot_{kriteria['nama']}"
            default_bobot =
st.session_state.bobot_kriteria.get(key_bobot, 0)

            bobot = st.number_input(
                f"Bobot {kriteria['nama']}",
                min_value=0,
                max_value=100,
                value=int(default_bobot),
                step=5,
                key=key_bobot,
                label_visibility="collapsed"
            )
            st.session_state.bobot_kriteria[key_bobot] = bobot

        with col3:
            # Kolom keterangan (read-only / disabled)
            if kriteria["jenis"] == "Cost":
                st.text_input(
                    "Keterangan",
                    value="Semakin kecil skala, semakin baik (Cost)",
                    disabled=True,
                    key=f"ket_{kriteria['nama']}",
                    label_visibility="collapsed"
                )
            else:
                st.text_input(
                    "Keterangan",

```

```

value="Semakin besar skala, semakin baik
(Benefit)",
                                disabled=True,
                                key=f"ket_{kriteria['nama']}",
                                label_visibility="collapsed"
                                )

st.divider()

# Hitung total bobot
total_bobot = sum(st.session_state.bobot_kriteria.values())

col_total1, col_total2, col_total3 = st.columns([1, 1, 2])
# Lebar Kolom
with col_total1:
    st.markdown("**Total Bobot:**")
with col_total2:
    if total_bobot == 100:
        st.success(f"{total_bobot}% ")
    elif total_bobot < 100:
        st.warning(f"{total_bobot}% (Kurang {100 -
total_bobot}%)")
    else:
        st.error(f"{total_bobot}% (Kelebihan {total_bobot -
100}%)")

with Tab5:
    selected_profile = st.session_state.get('selected_profile')
    pilihan_perusahaan = st.session_state.get('pilihan_perusahaan')
    skala_kriteria = st.session_state.get('skala_kriteria', {})
    bobot_kriteria = st.session_state.get('bobot_kriteria', {})

    if selected_profile is None or selected_profile == "Semua Lowongan":
        st.warning("Harap Memilih Lowongan Magang Terlebih Dahulu Di Tab
1")
    elif not pilihan_perusahaan or len(pilihan_perusahaan) == 0:
        st.warning("Harap Memilih Alternatif Perusahaan Terlebih Dahulu
Di Tab 2")
    elif not skala_kriteria:
        st.warning("Harap Mengisi Skala Kriteria Terlebih Dahulu Di Tab
3")
    elif sum(bobot_kriteria.values()) != 100:
        st.warning("Harap Mengisi Bobot Kriteria dengan total 100% Di Tab
4")
    else:
        kriteria_list = [
            {"nama": "Location", "jenis": "Cost"},
            {"nama": "Stipend", "jenis": "Benefit"},
            {"nama": "Duration", "jenis": "Benefit"},
            {"nama": "Perks", "jenis": "Benefit"},
            {"nama": "Tipe Kerja", "jenis": "Benefit"}
        ]

        # 1. Matriks Keputusan (X) dengan NumPy
        jumlah_alternatif = len(pilihan_perusahaan)
        jumlah_kriteria = len(kriteria_list)

        # Buat matriks X ukuran (jumlah_alternatif x jumlah_kriteria)
        X = np.zeros((jumlah_alternatif, jumlah_kriteria))

```

```

for i, perusahaan in enumerate(pilihan_perusahaan):
    for j, kriteria in enumerate(kriteria_list):
        nama_kriteria = kriteria['nama']
        key = f"skala_{nama_kriteria}_{perusahaan}"
        X[i, j] = skala_kriteria.get(key, 0)

# 2. Normalisasi Matriks (R) dengan NumPy (VEKTORISASI)
R = np.zeros_like(X, dtype=float)

for j, kriteria in enumerate(kriteria_list):
    kolom = X[:, j] # Ambil kolom ke-j (satu kriteria)
    jenis = kriteria['jenis']

    if jenis == "Benefit":
        max_val = np.max(kolom)
        if max_val != 0:
            R[:, j] = kolom / max_val
        else:
            R[:, j] = 0
    else: # Cost
        min_val = np.min(kolom)
        if min_val != 0:
            R[:, j] = min_val / kolom
            # Handle division by zero (jika ada nilai 0)
            R[:, j] = np.where(kolom == 0, 0, R[:, j])
        else:
            R[:, j] = 0

# 3. Hitung Bobot dalam bentuk NumPy array
bobot_array = np.zeros(jumlah_kriteria)
for j, kriteria in enumerate(kriteria_list):
    nama_kriteria = kriteria['nama']
    key_bobot = f"bobot_{nama_kriteria}"
    bobot_array[j] = bobot_kriteria.get(key_bobot, 0) / 100

# 4. Hitung Nilai Preferensi (V) dengan perkalian matriks
# V = R × bobot (perkalian baris dengan bobot)
nilai_preferensi = np.sum(R * bobot_array, axis=1)

# 5. Buat hasil_saw dengan detail perhitungan
hasil_saw = []
for i, perusahaan in enumerate(pilihan_perusahaan):
    total_nilai = nilai_preferensi[i]
    detail_perhitungan = {}

    for j, kriteria in enumerate(kriteria_list):
        nama_kriteria = kriteria['nama']
        detail_perhitungan[nama_kriteria] = {
            'nilai_normalisasi': round(R[i, j], 4),
            'bobot': round(bobot_array[j], 4),
            'kontribusi': round(R[i, j] * bobot_array[j], 4)
        }

    hasil_saw.append({
        'Company Name': perusahaan,
        'Hasil Perhitungan': round(total_nilai, 4),
        'Detail': detail_perhitungan
    })

```

```

# 6. Untuk Matriks Normalisasi (ke df_normalisasi)
df_normalisasi = pd.DataFrame(
    R.round(4), # Bulatkan ke 4 desimal
    index=pilihan_perusahaan,
    columns=[k['nama'] for k in kriteria_list]
)
# Tambahkan baris jenis kriteria
df_normalisasi.loc['Jenis'] = [k['jenis'] for k in kriteria_list]

df_hasil = pd.DataFrame(hasil_saw)
df_hasil = df_hasil.sort_values(by='Hasil Perhitungan',
ascending=False).reset_index(drop=True)
df_hasil.index = df_hasil.index + 1
df_hasil.reset_index(inplace=True)
df_hasil.rename(columns={'index': 'Ranking'}, inplace=True)

# Matriks Normalisasi
st.subheader("Matriks Normalisasi (R)")
st.caption("Nilai hasil normalisasi dari skala kriteria (Benefit:
max→1, Cost: min→1)")
st.dataframe(df_normalisasi, use_container_width=True)

# Detail Perhitungan
st.subheader("Detail Perhitungan Nilai Akhir (V)")
st.caption("Nilai akhir =  $\Sigma$  (Nilai Normalisasi  $\times$  Bobot)")

for perusahaan in pilihan_perusahaan:
    data_perusahaan = next((item for item in hasil_saw if
item['Company Name'] == perusahaan), None)
    if data_perusahaan:
        st.markdown(f"***Alternatif: {perusahaan}***")

        detail_data = []
        for kriteria in kriteria_list:
            nama_kriteria = kriteria['nama']
            det = data_perusahaan['Detail'][nama_kriteria]
            detail_data.append({
                'Kriteria': nama_kriteria,
                'Jenis': kriteria['jenis'],
                'Nilai Normalisasi (R)':
det['nilai_normalisasi'],
                'Bobot (W)': det['bobot'],
                'Kontribusi (R $\times$ W)': det['kontribusi']
            })

        df_detail = pd.DataFrame(detail_data)
        st.dataframe(df_detail, use_container_width=True,
hide_index=True)
        st.markdown(f"***Total Nilai Alternatif:
{data_perusahaan['Hasil Perhitungan']}***")
        st.divider()

        st.subheader("Hasil Peranki ngan Alternatif Terbaik (Metode
SAW)")
        st.dataframe(df_hasil[['Ranking', 'Company Name', 'Hasil
Perhitungan']],
                    use_container_width=True, hide_index=True)

```

```

# Kesimpulan
if not df_hasil.empty:
    terbaik = df_hasil.iloc[0]
    company_terbaik = terbaik['Company Name']
    nilai_terbaik = terbaik['Hasil Perhitungan']

    # Ambil stipend dari dataset
    df_alternatif = filtered_df_Tab1[filtered_df_Tab1['company']
== company_terbaik]
    stipend_terbaik = "N/A"
    if not df_alternatif.empty:
        stipend_terbaik = df_alternatif['Stipend'].iloc[0]

    st.success(f"Kesimpulan: Tempat magang terbaik adalah
**{company_terbaik}** dengan nilai SAW **{nilai_terbaik}** dan estimasi
gaji {stipend_terbaik}")

# Simpan hasil ke session_state untuk tab visualisasi
st.session_state.df_hasil_saw = df_hasil
st.session_state.matriks_normalisasi = df_normalisasi

with Tab6:
    selected_profile = st.session_state.get('selected_profile')
    pilihan_perusahaan = st.session_state.get('pilihan_perusahaan')
    df_hasil_saw = st.session_state.get('df_hasil_saw')

    if selected_profile is None or selected_profile == "Semua Lowongan":
        st.warning("Harap Memilih Lowongan Magang Terlebih Dahulu Di Tab
1")
    elif not pilihan_perusahaan or len(pilihan_perusahaan) == 0:
        st.warning("Harap Memilih Alternatif Perusahaan Terlebih Dahulu
Di Tab 2")
    elif df_hasil_saw is None or df_hasil_saw.empty:
        st.warning("Harap Menyelesaikan Perhitungan Di Tab 5 Terlebih
Dahulu")
    else:
        st.subheader("Visualisasi Hasil Perhitungan SAW")

        df_plot = df_hasil_saw.sort_values('Hasil Perhitungan',
ascending=True)
        companies = df_plot['Company Name'].tolist()
        scores = df_plot['Hasil Perhitungan'].tolist()

        # 1. Grafik (Line Chart)
        st.markdown("**Grafik (Line Chart)**")
        fig1, ax1 = plt.subplots(figsize=(10, 5))
        ax1.plot(df_hasil_saw['Company Name'], df_hasil_saw['Hasil
Perhitungan'], marker='o', linestyle='-', color='b')
        ax1.set_xlabel('Company Name')
        ax1.set_ylabel('Hasil Perhitungan')
        ax1.set_title('Grafik Hasil Perhitungan SAW')
        ax1.grid(True, linestyle='--', alpha=0.7)
        plt.xticks(rotation=45, ha='right')
        st.pyplot(fig1)

        # 2. Bar Chart
        st.markdown("**Bar Chart**")
        fig2, ax2 = plt.subplots(figsize=(10, 5))
        ax2.barh(companies, scores, color='skyblue')

```

```

ax2.set_xlabel('Hasil Perhitungan')
ax2.set_ylabel('Company Name')
ax2.set_title('Bar Chart Hasil Perhitungan SAW')
st.pyplot(fig2)

# 3. Pie Chart
st.markdown("***Pie Chart***")
fig3, ax3 = plt.subplots(figsize=(8, 8))
ax3.pie(df_hasil_saw['Hasil Perhitungan'],
labels=df_hasil_saw['Company Name'], autopct='%1.1f%%', startangle=90)
ax3.axis('equal') # Equal aspect ratio ensures that pie is drawn
as a circle.
ax3.set_title('Pie Chart Proporsi Hasil Perhitungan SAW')
st.pyplot(fig3)

```

Source Code Style.py

```

import streamlit as st

# Alternatif untuk mengatur pilihan alternatif
# Value untuk mengatur i+1

def visualisasi():
    return """
<style>

.kolom{
width : 100%;
height : 50px;
background-color : #262730;
padding : 12px;
border-radius : 10px
}

"""

def hasilData(alternatif, label, value):
    if alternatif == "(belum dipilih)":
        st.markdown(
            f"""
<p style='font-size: 13px; margin-bottom:
10px;'>{label}{value}</p>
<p class="kolom" style="color: #FFFFFF;">Belum Dipilih</p>
""",
            unsafe_allow_html=True
        )
    else:
        st.markdown(
            f"""
<p style='font-size: 13px; margin-bottom:
10px;'>{label}{value}</p>
<p class="kolom" style="color: #FFFFFF;">{alternatif}</p>
""",
            unsafe_allow_html=True
        )

def hasilDataBaru(alternatif, label, value):
    st.markdown(

```



```
f"""
    <p style='font-size: 13px; margin-bottom:
10px;'>{label}{value}</p>
    <p class="kolom" style="color: #FFFFFF;">{alternatif}</p>
    """,
    unsafe_allow_html=True
)
```

1. Memilih Lowongan Magang



2. Memilih Jumlah Alternatif dan Alternatif Perusahaan



3. Penentuan Skala Kriteria Berdasarkan Alternatif

(Location , Stipend , Duration , Perks , Tipe Kerja)

Deploy

Program Pengambilan Keputusan Tempat Magang Terbaik

Metode SAW

Anggota Kelompok :

1. Yohanes Herang Aji Dharma || 123240191
2. Alvin Andhika Putra || 123240193

Pilih Lowongan Magang Alternatif **Skala Kriteria** Pembobotan Kriteria Hasil Alternatif Terbaik Visualisasi

Penentuan Skala Kriteria

Kriteria	Alternatif 1	Alternatif 2	Alternatif 3
Location Splashgain Technology Solutions Private Limited: Pune Icecreamlabs: Bangalore Entrepreneur Growth Lab (OPC) Private Limited: Work from home	1	5	10
Stipend Splashgain Technology Solutions Private Limited: ₹ 8,000 - 15,000 /month Icecreamlabs: ₹ 20,000 - 30,000 /month Entrepreneur Growth Lab (OPC) Private Limited: ₹ 12,000 - 18,000 /month	1	10	5
Duration Splashgain Technology Solutions Private Limited: 6 Months Icecreamlabs: 3 Months Entrepreneur Growth Lab (OPC) Private Limited: 3 Months	10	5	5
Perks Splashgain Technology Solutions Private Limited: Certificate, Flexible, Letter Of Recommendation, 5 Days A Week Icecreamlabs: Mentorship Entrepreneur Growth Lab (OPC) Private Limited: Certificate, Flexible, Letter Of Recommendation	10	1	5
Tipe Kerja Splashgain Technology Solutions Private Limited: WFO Icecreamlabs: WFO Entrepreneur Growth Lab (OPC) Private Limited: WFH	7	7	10

4. Pembobotan Kriteria

Deploy

Program Pengambilan Keputusan Tempat Magang Terbaik

Metode SAW

Anggota Kelompok :

- Yohanes Herang Aji Dharma || 123240191
- Alvin Andhika Putra || 123240193

Pilih Lowongan Magang

Alternatif

Skala Kriteria

Pembobotan Kriteria

Hasil Alternatif Terbaik

Visualisasi

Atur bobot untuk setiap kriteria (total harus 100%)

Location

30

-

+

Semakin kecil skala, semakin baik (Cost)

Stipend

30

-

+

Semakin besar skala, semakin baik (Benefit)

Duration

10

-

+

Semakin besar skala, semakin baik (Benefit)

Perks

10

-

+

Semakin besar skala, semakin baik (Benefit)

Tipe Kerja

20

-

+

Semakin besar skala, semakin baik (Benefit)

Total Bobot:

100%

5. Hasil Alternatif Terbaik

Program Pengambilan Keputusan Tempat Magang Terbaik

Metode SAW

Anggota Kelompok :

- Yohanes Herang Aji Dharma || 123240191
- Alvin Andhika Putra || 123240193

Pilih Lowongan Magang

Alternatif

Skala Kriteria

Pembobotan Kriteria

Hasil Alternatif Terbaik

Visualisasi

Matriks Normalisasi (R)

Nilai hasil normalisasi dari skala kriteria (Benefit: max+1, Cost: min+1)

	Location	Stipend	Duration	Perks	Tipe Kerja
Splashgain Technology Solutions Private Limited	1.0	0.1	1.0	1.0	0.7
Iccreamlabs	0.2	1.0	0.5	0.1	0.7
Entrepreneur Growth Lab (OPC) Private Limited	0.1	0.5	0.5	0.5	1.0
Jenis	Cost	Benefit	Benefit	Benefit	Benefit

Detail Perhitungan Nilai Akhir (V)

Nilai akhir = $\sum$ (Nilai Normalisasi $\times$ Bobot)

Alternatif: Splashgain Technology Solutions Private Limited

Kriteria	Jenis	Nilai Normalisasi (R)	Bobot (W)	Kontribusi (R $\times$ W)
Location	Cost		1	0.3
Stipend	Benefit		0.1	0.3
Duration	Benefit		1	0.1
Perks	Benefit		1	0.1
Tipe Kerja	Benefit		0.7	0.2

Total Nilai Alternatif: 0.67

Alternatif: Iccreamlabs

Kriteria	Jenis	Nilai Normalisasi (R)	Bobot (W)	Kontribusi (R $\times$ W)
Location	Cost		0.2	0.3
Stipend	Benefit		1	0.3
Duration	Benefit		0.5	0.1
Perks	Benefit		0.1	0.1
Tipe Kerja	Benefit		0.7	0.2

Total Nilai Alternatif: 0.56

Alternatif: Entrepreneur Growth Lab (OPC) Private Limited

Kriteria	Jenis	Nilai Normalisasi (R)	Bobot (W)	Kontribusi (R*W)	
Location	Cost		0.1	0.3	0.03
Stipend	Benefit		0.5	0.3	0.15
Duration	Benefit		0.5	0.1	0.05
Perks	Benefit		0.5	0.1	0.05
Tipe Kerja	Benefit		1	0.2	0.2

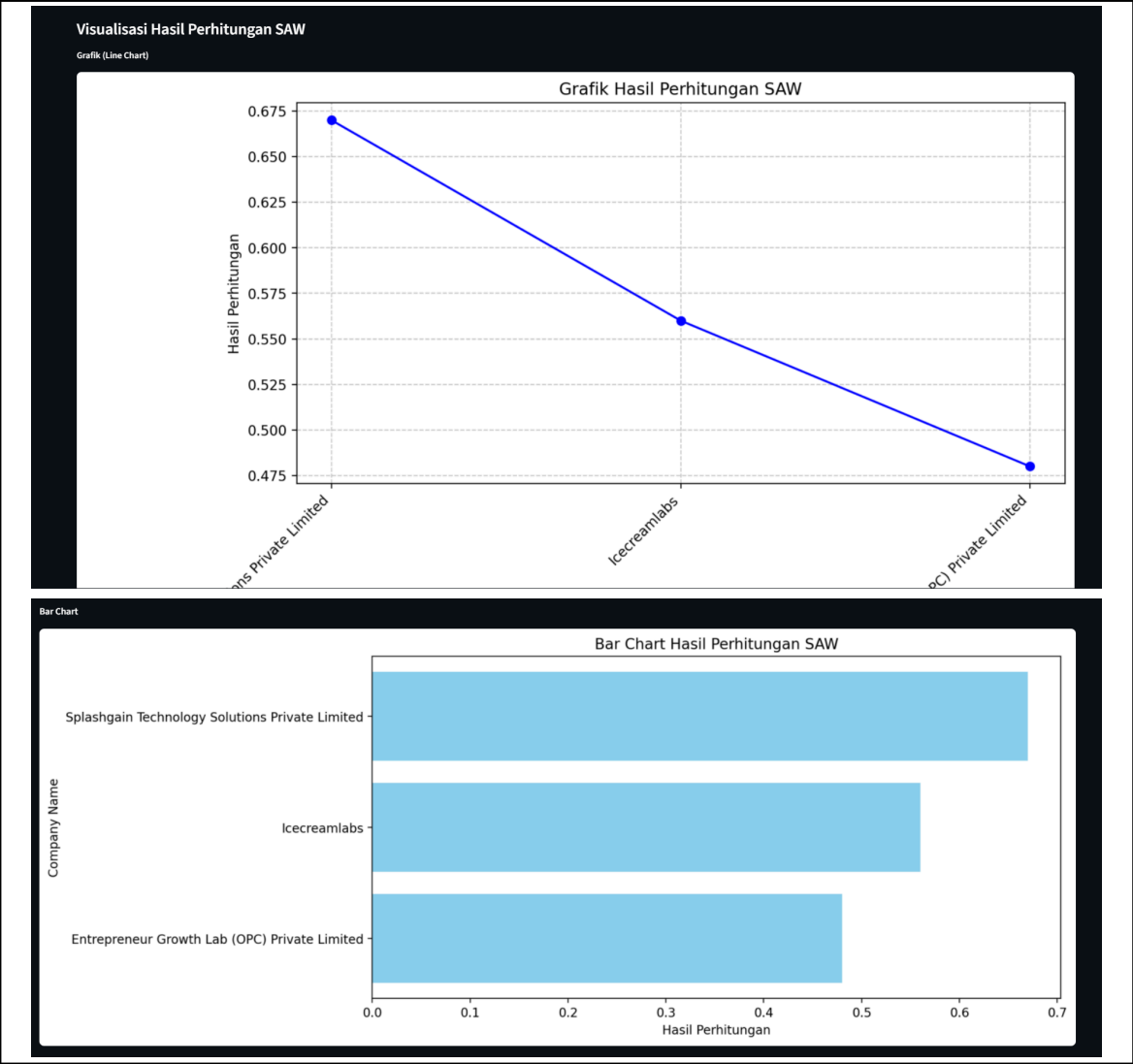
Total Nilai Alternatif: 0.48

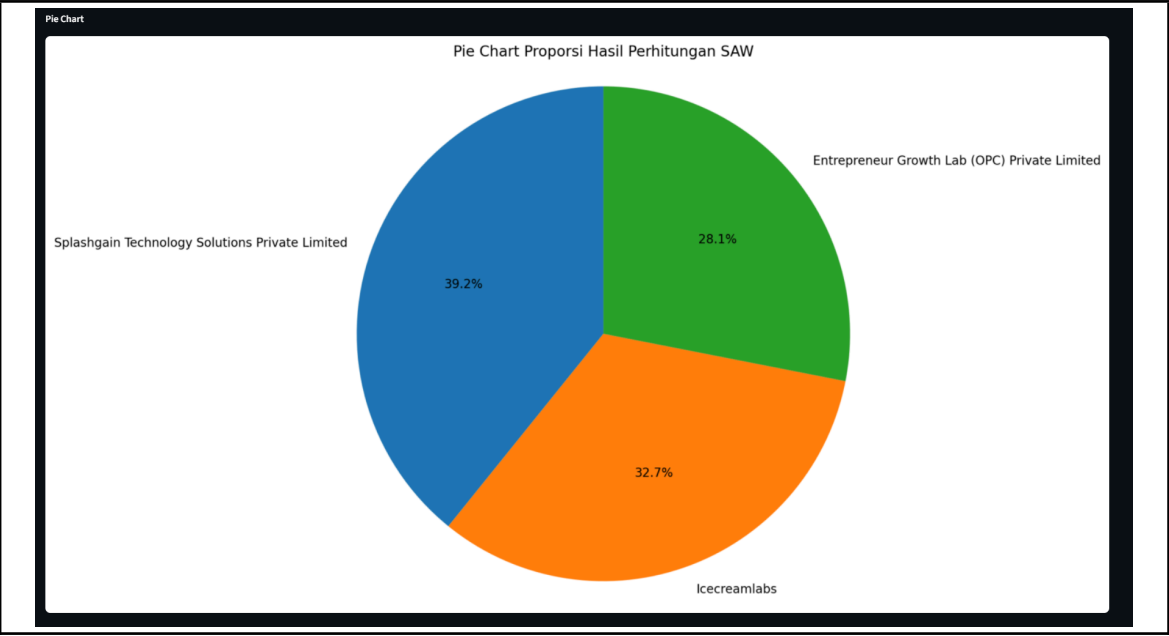
Hasil Perankingan Alternatif Terbaik (Metode SAW)

Ranking	Company Name	Hasil Perhitungan
1	Splashgain Technology Solutions Private Limited	0.67
2	Icecreamlabs	0.56
3	Entrepreneur Growth Lab (OPC) Private Limited	0.48

Kesimpulan: Tempat magang terbaik adalah Splashgain Technology Solutions Private Limited dengan nilai SAW 0.67 dan estimasi gaji ₹ 8,000 - 15,000 /month

6. Visualisasi





BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026									
		Mei									
		1	2	3	4	5	6	7	8	9	10
1	Mencari ide dan goals yang akan dicapai dengan tema yang sudah ditentukan										
2	Mencari dataset program magang										
3	Merancang tampilan										
4	Revisi dataset										
5	Membuat fungsi pada menu atau tab 1 , 2 , dan 3										
6	Membuat fungsi pada menu atau tab 4,5,6										
7	Merapikkan tampilan dengan style.py										
8	Test Program										
9	Laporan										

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Yohanes Herang Aji Dharma	123240191	- Mengerjakan Tab 4 - Mengerjakan Tab 5 - Mengerjakan Tab 6 - Mengerjakan Tab 1 - Laporan

			- Error Handling
2	Alvin Andhika Putra	123240193	- Mengerjakan Tab 2 - Mengerjakan Tab 3 - Mengerjakan Tab 5 - Laporan - Mencari Dataset - Menyusun Tampilan Web

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan dan implementasi sistem pendukung keputusan pemilihan tempat magang terbaik menggunakan metode Simple Additive Weighting (SAW), dapat disimpulkan bahwa sistem yang dibuat mampu membantu pengguna dalam menentukan alternatif tempat magang secara lebih efektif dan terstruktur. Sistem ini memanfaatkan beberapa kriteria penilaian seperti lokasi, stipend, durasi magang, fasilitas (perks), dan tipe kerja untuk menghasilkan rekomendasi perusahaan terbaik sesuai preferensi pengguna.

Melalui proses normalisasi dan pembobotan pada metode SAW, sistem dapat menghitung nilai preferensi dari setiap alternatif perusahaan sehingga menghasilkan perankingan yang objektif. Implementasi sistem menggunakan Python dan Streamlit juga berhasil memberikan tampilan antarmuka yang interaktif, sehingga pengguna dapat memilih lowongan, menentukan skala penilaian, memberikan bobot pada setiap kriteria, hingga melihat hasil visualisasi perhitungan dengan mudah.

Selain itu, sistem mampu menampilkan hasil akhir berupa rekomendasi perusahaan magang terbaik berdasarkan nilai tertinggi dari proses perhitungan SAW. Dengan adanya sistem ini, proses pengambilan keputusan yang sebelumnya dilakukan secara manual menjadi lebih cepat, akurat, dan efisien. Oleh karena itu, proyek ini berhasil memenuhi tujuan utama yaitu membangun sistem pendukung keputusan untuk membantu mahasiswa dalam memilih tempat magang terbaik sesuai kebutuhan dan prioritas masing-masing.

4.3 Saran

Sistem yang telah dibuat masih dapat dikembangkan lebih lanjut agar memiliki fitur dan kemampuan yang lebih baik. Salah satu pengembangan yang dapat dilakukan adalah menambahkan metode perhitungan lain sehingga pengguna dapat membandingkan hasil rekomendasi dari beberapa metode pengambilan keputusan.

Selain itu, sistem dapat dikembangkan dengan menambahkan fitur login dan penyimpanan data pengguna agar hasil penilaian dapat disimpan dan digunakan kembali di kemudian hari. Pengembangan pada sisi database juga dapat dilakukan supaya data lowongan magang dapat diperbarui secara otomatis dan lebih terintegrasi dengan sumber data eksternal.

Dari sisi antarmuka, visualisasi data dapat dibuat lebih menarik dan interaktif agar pengguna lebih mudah memahami hasil perankingan. Sistem juga dapat dikembangkan menjadi aplikasi berbasis web yang lebih responsif atau aplikasi mobile sehingga dapat diakses dengan lebih fleksibel. Dengan adanya pengembangan tersebut, diharapkan sistem pendukung keputusan ini dapat memberikan hasil yang lebih optimal dan bermanfaat bagi pengguna dalam menentukan tempat magang terbaik.

DAFTAR PUSTAKA

Nath, J. (2025). *Internship Opportunities in India (2025)* [Dataset]. Kaggle.

Diambil dari

<https://www.kaggle.com/datasets/jayaantanaath/internship-opportunities-in-india-2025>